Independent University, Bangladesh

Compiler Quiz 02 (LL(1) Parsing) · Autumn 2026 · Total marks 15 · Time 20 min

Question paper

1. Write the First() and Follow() of the following Grammar. Draw LL(1) Parsing Table and parse the string `int id , id ;` showing all steps. If parsing is not possible, point out the reasons. Here, Non-terminals = {D, T, L, L'}, Terminals = {;, int, char, id, ,, ε}, and Start symbol = {D}. **[15]**

```
D → T L ;
T → int | char
L → id L'
L' → , id L' | ε
```

Answers (exam layout, with comments)

Answer 1 [15]

WRITE THIS

Numbered rules: 1. D → T L ; 2. T → int 3. T → char 4. L → id L' 5. L' → , id L' 6. L' → ε

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
D	{ int, char }	{ $ }
T	{ int, char }	{ id }
L	{ id }	{ ; }
L'	{ ,, ε }	{ ; }

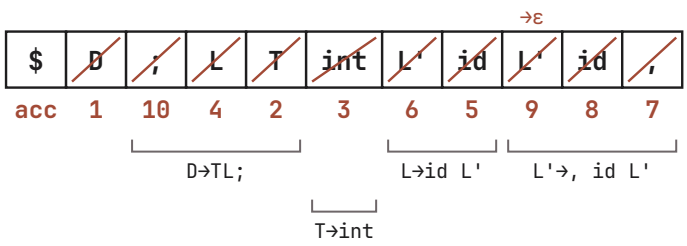
Step 2 · LL(1) parsing table

	;	int	char	id	,	$
D		D→TL;	D→TL;			
T		T→int	T→char			
L				L→id L'		
L'	L'→ε				L'→, id L'	

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3 · Parse `int id , id ;` (horizontal stack)

Buffer: `int id , id ; $` LA ↑ starts at `int`



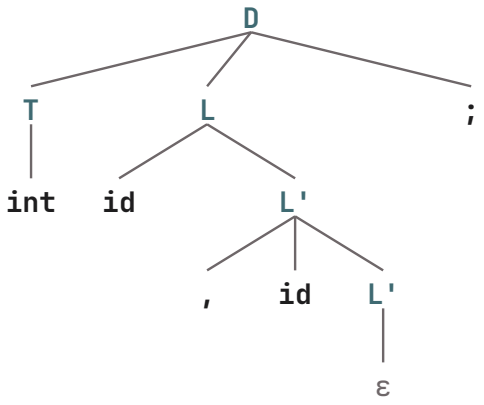
How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop D (LA = int): D→TL;, push ; L T
2. pop T (LA = int): T→int, push int
3. match int, pop it, move LA → id
4. pop L (LA = id): L→id L', push L' id
5. match id, pop it, move LA → ,
6. pop L' (LA = ,): L'→, id L', push L' id ,
7. match ,, pop it, move LA → id
8. match id, pop it, move LA → ;
9. pop L' (LA = ;): L'→ε, push nothing
10. match ;, pop it, move LA → $

acc. Top = $ and LA = $ → accept

Parse tree



Leaves left to right (skip ε): `int id , id ;` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(D): D → T L ; gives { int, char }. Total { int, char }.
- FIRST(T): T → int gives { int }. T → char gives { char }. Total { int, char }.
- FIRST(L): L → id L' gives { id }. Total { id }.
- FIRST(L'): L' → , id L' gives { , }. L' → ε gives ε. Total { ,, ε }.
- FOLLOW(D): start symbol → add $. Total { $ }.
- FOLLOW(T): in D → T L ;: after T is L ; → add { id }. Total { id }.
- FOLLOW(L): in D → T L ;: after L is ; → add { ; }. Total { ; }.
- FOLLOW(L'): in L → id L': nothing after L' → add FOLLOW(L). Total { ; }.

Lose marks by: FOLLOW(L') must be { ; } (from L → id L', it gets FOLLOW(L)); missing the L' → ε entry under ;.